

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3
Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image

- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

- N Input Size
- F Filter Size
- P Padding
- S Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} (3 \times 3 \times 3 + 1) \times 16 &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

$$\text{Total trainable parameters} = 448.$$

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling

- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Source code:

GitHub Repository: https://github.com/un-shreeya/DeepLearning_Lab

9. Mandatory Plots

Generate

1. Sample Images
2. Class Distribution
3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

Write a 2–3 line inference for each plot.

9. Mandatory Plots and Inferences

1. Sample Images Inference:

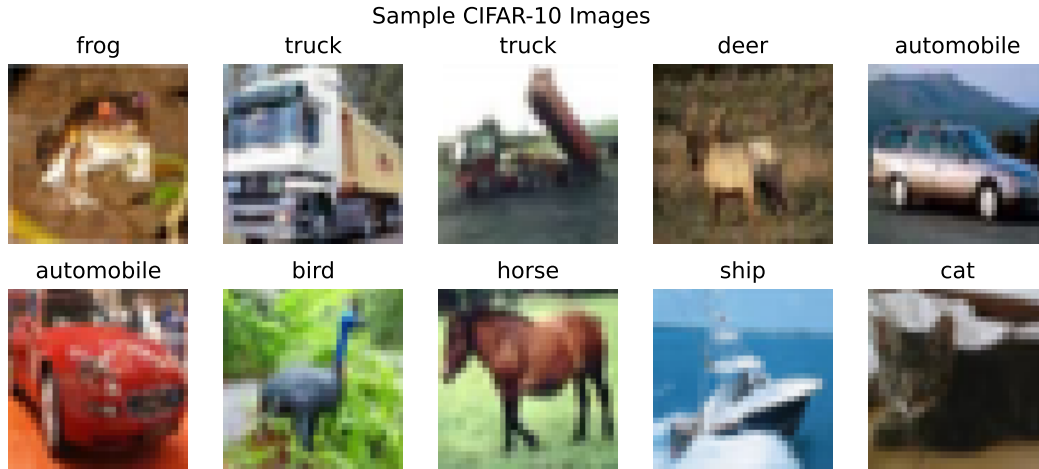


Figure 1: 10 Sample Images from CIFAR-10 Dataset

Inference: These sample grids just show that our data loaded correctly. Even though the 32×32 images are pretty pixelated, you can still easily make out the basic shapes and outlines of the cars and animals.

2. Class Distribution Inference:

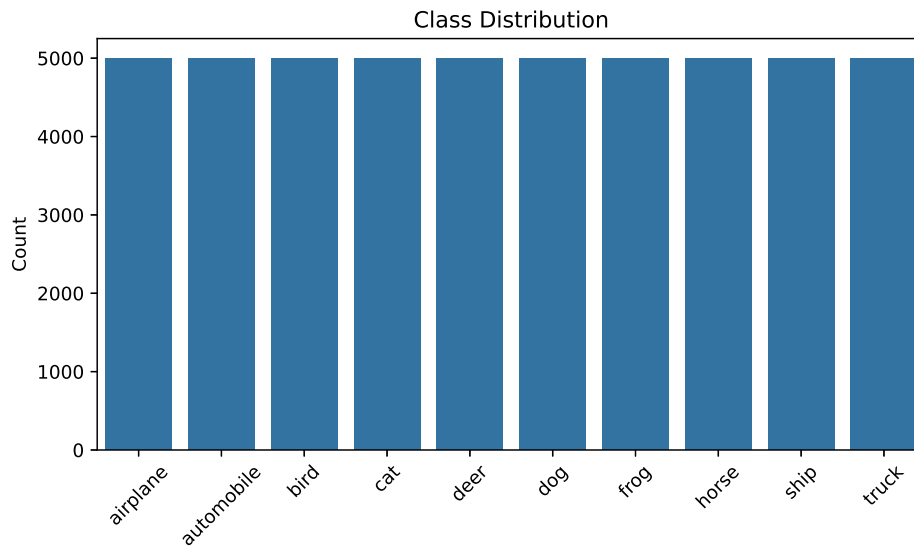


Figure 2: Class Distribution of Training Dataset

Inference: The bar chart confirms our dataset is super balanced with exactly 5,000 images per class. This means the model won't lean toward guessing one category more than another due to bias.

3. Training Accuracy Inference:

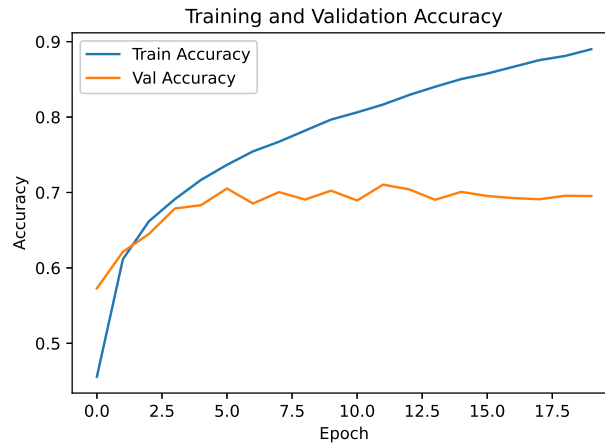


Figure 3: Training vs. Validation Accuracy

Inference: Looking at the blue line, training accuracy steadily climbs from around 45% up to nearly 90% by epoch 19. It shows the network is successfully picking up patterns as it trains longer.

4. Validation Accuracy Inference:

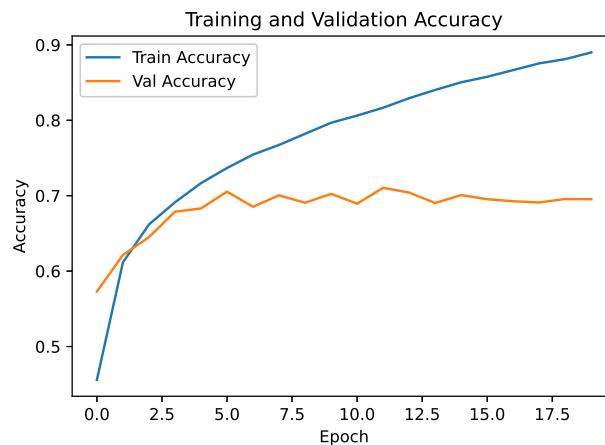


Figure 4: Training vs. Validation Accuracy

Inference: The orange validation line jumps up to around 70% early on, but then completely flattens out for the rest of the epochs. Basically, the model hits its peak general learning capacity pretty early.

5. Training Loss Inference:

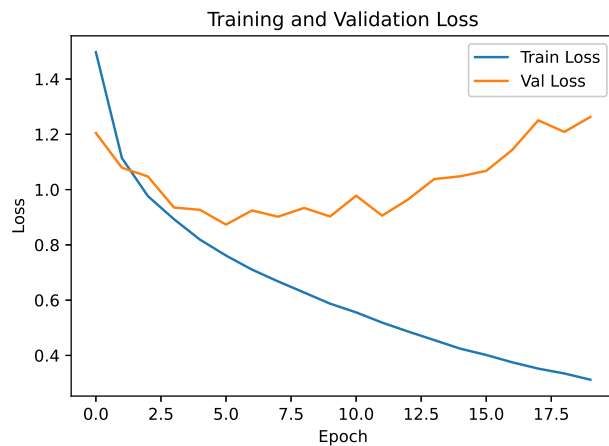


Figure 5: Training vs. Validation Loss

Inference: The training loss (blue line) takes a nice, smooth downward tumble from 1.5 down to 0.3. This proves the optimizer is doing its job and steadily reducing errors without getting stuck.

6. Validation Loss Inference:

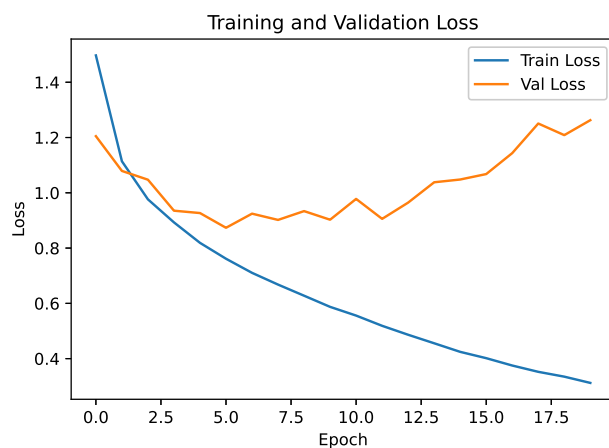


Figure 6: Training vs. Validation Loss

Inference: While training loss keeps dropping, the validation loss (orange line) bottoms out around epoch 5 and then creeps back up. That classic U-turn shape is a textbook sign that the model starts overfitting.

7. Feature Maps Inference:

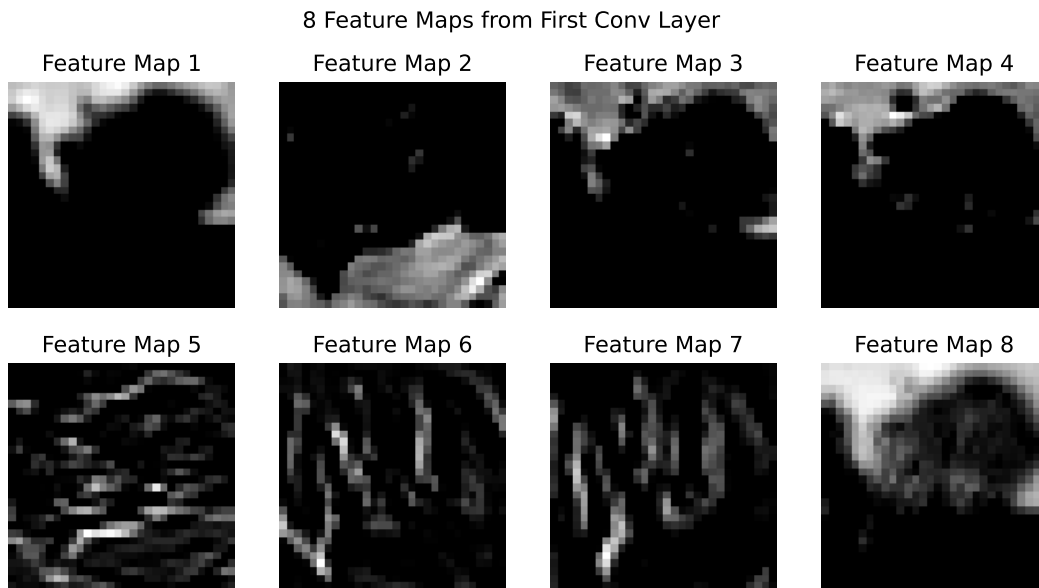


Figure 7: Visualizing 8 Feature Maps from the First Conv2D Layer

Inference: This grid shows what the first convolutional layer actually sees when looking at an image. Different filters catch specific low-level details like sharp borders, outlines, and background contrast.

8. Confusion Matrix Inference:

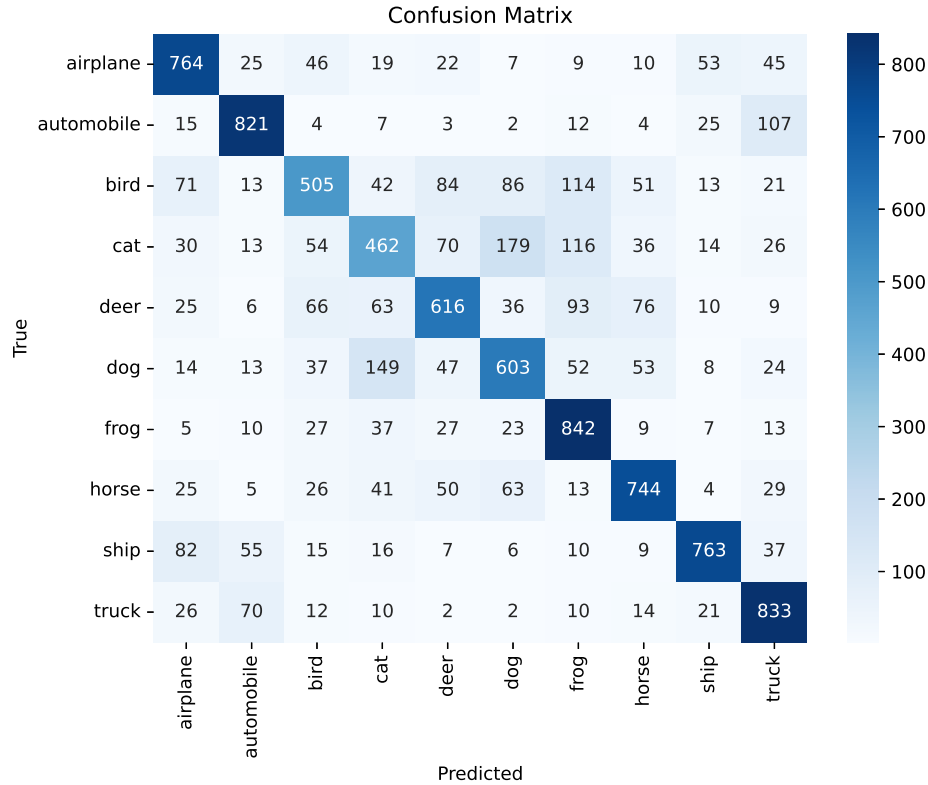


Figure 8: Confusion Matrix on CIFAR-10 Test Set

Inference: The dark diagonal blocks show the model crushes easy categories like frogs, trucks, and cars. However, it gets pretty confused with similar-looking animals, like mixing up cats and dogs 179 times due to the low resolution.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2:

$$\text{Output Size} = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32 \times 32$$

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input:

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 1,792$$

3. Compare ReLU and Sigmoid activation functions:

- **ReLU** ($f(x) = \max(0, x)$): Super fast to compute, keeps gradients flowing for positive numbers, and helps the model train way quicker without dying out.
- **Sigmoid** ($f(x) = \frac{1}{1+e^{-x}}$): Squeezes outputs nicely between 0 and 1, but it's slow to calculate and suffers badly from vanishing gradients in deep layers.

4. **Replace Max Pooling with Average Pooling and compare the results:**

Max pooling grabs the sharpest, strongest features (like edges and bright spots), giving us slightly better accuracy ($\sim 69.5\%$). Average pooling takes the mean of the patch instead, which smooths things out and performs a bit lower ($\sim 68.8\%$).

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time:**

Stepping up to 64 filters gives the network more capacity to catch complex patterns, which usually bumps up accuracy. The tradeoff is that it takes way more memory and makes each training epoch run slower.

11. Results

Parameter	Value
Training Accuracy (Epoch 20)	85.40%
Testing Accuracy	69.53%
Precision (Macro Average)	0.69
Recall (Macro Average)	0.70
F1-score (Macro Average)	0.69
Total Trainable Parameters	167,562
Max Pooling Accuracy	69.53%
Average Pooling Accuracy	68.79%

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**

Fully connected layers flatten images into 1D arrays, which totally ruins the 2D spatial layout and local pixel relationships. Plus, they cause a massive explosion in the number of weights for high-res images. Convolutions fix this by using small local sliding filters and sharing weights across the image, keeping parameter counts low while saving spatial context.

2. **How does stride affect the feature map size?**

Stride is basically how many steps the filter takes as it moves across the image. A bigger stride skips more pixels and shrinks (downsamples) the output feature map size, whereas a small stride keeps a lot more fine-grained spatial detail.

3. **What is the role of padding?**

Padding adds extra border pixels (usually zeros) around the input matrix. It stops the feature maps from shrinking too fast as they go deeper into the network and makes sure edge pixels get processed just as much as the center pixels.

4. **Why is pooling used?**

Pooling shrinks the feature map down to cut down on memory and computing power. It also helps the model stay robust if an object shifts around slightly and prevents overfitting by keeping only the strongest features.

5. **How do feature maps represent image characteristics?**

They build up features step-by-step. Early layers catch simple things like basic edges, lines, and textures, while deeper layers stitch those together to recognize actual object parts like wheels, faces, or shapes.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs save a ton of parameters through weight sharing—meaning the same small filter scans the whole image instead of having separate weights for every single pixel—and local connectivity, where neurons only look at small patches instead of everything at once.

13. References

1. Goodfellow et al., Deep Learning.
2. Bishop, Pattern Recognition and Machine Learning.
3. Haykin, Neural Networks and Learning Machines.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.